CENTRIX — Phased Technical, Security & Detection Audit

Audit baseline, error analysis, false-positive/false-negative investigation, remediation and final validation plan

Repository: github.com/harshraj211/CENTRIX
Audit status: Phase plan prepared; source-code execution audit pending attachment of the extracted ZIP
Primary objective: determine what CENTRIX actually implements, what is incorrect, what is missing, and how to make the scanner evidence-driven and reliable.

1. Executive Summary

CENTRIX is intended as an authorised Dynamic Application Security Testing (DAST) platform. The documented project scope includes target validation, controlled crawling, vulnerability checks, finding enrichment, persistence and report generation. Existing project material also describes more advanced capabilities such as Playwright-based SPA analysis, ChromaDB/RAG, multi-agent reasoning and a broader scanner set. These advanced claims must be reconciled against the actual submitted source code before they are treated as implemented.

The audit will therefore use a strict source-of-truth approach: code and reproducible test results take precedence over README statements, diagrams and research-paper claims. Every reported finding should ultimately have evidence, verification status, confidence, classification and a reproducible test case.

Important: this document is the phased audit/remediation blueprint. The actual source-code findings, line-level bugs and measured false-positive/false-negative rates will be populated after the extracted CENTRIX ZIP is attached and inspected.

2. Audit Principles

Principle	Rule
Source of truth	Executable/source implementation and reproducible tests outrank documentation claims.
Safety	Testing must use owned, authorised or intentionally vulnerable targets.
Evidence	A scanner finding is not considered verified merely because a payload was sent or reflected.
False positive control	Require contextual evidence and, where possible, a confirmation probe.
False negative control	Use known-vulnerable applications and deterministic regression cases.
Severity integrity	CWE/CVSS must match the actual demonstrated condition, not a generic payload label.
Traceability	Every finding should be traceable from target → request → response → detector → evidence → report.

3. Phase Overview

Phase	Focus	Primary outcome
1	Baseline & Architecture	Verified inventory and real execution flow
2	Build & Runtime Reliability	Reproducible installation and clean execution
3	Scanner-Core Correctness	Correct target handling, crawling and request pipeline
4	False-Positive Reduction	Evidence-backed finding verification
5	False-Negative Reduction	Expanded coverage and detection regression suite

6	Finding Intelligence	Correct CWE/CVSS/confidence/deduplication
7	ML/RAG/AI Verification	Prove which intelligence features are real and useful
8	Reporting & Persistence	Consistent, reproducible and auditable reports
9	Security Hardening	Secure CENTRIX itself and enforce scan boundaries
10	Performance & Concurrency	Controlled resource use and predictable scanning
11	End-to-End Validation	Benchmark against authorised vulnerable targets
12	Release Readiness	Final acceptance gate and technical documentation

Phase 1 — Baseline & Architecture Verification

• Inventory every directory, service, dependency, scanner module, API endpoint, database model and report generator.

• Trace the actual lifecycle: target input → validation → discovery → crawl → probes → finding creation → enrichment → storage → report.

• Compare implementation against README, architecture PDFs and research-paper claims.

• Identify dead code, duplicate implementations, unreachable modules and features described but not wired into the runtime.

• Deliverable: verified architecture diagram, feature matrix and discrepancy register.

Phase 2 — Build & Runtime Reliability

• Install from a clean environment and execute the documented setup procedure.

• Check Python/Node dependency compatibility, environment variables, Docker services, database initialization and startup order.

• Run backend, frontend and scanner independently and together.

• Record crashes, unhandled exceptions, timeouts, dependency failures and configuration assumptions.

• Deliverable: reproducible setup procedure and P0/P1 runtime defect list.

Phase 3 — Scanner-Core Correctness

• Audit target parsing, URL normalization, redirect handling, DNS resolution, IP classification and scope enforcement.

• Audit crawler behaviour: duplicate URLs, query parameters, forms, JavaScript routes, SPA content, robots/sitemap handling where relevant.

• Audit asynchronous request scheduling, concurrency limits, timeouts, retries and cancellation.

• Verify that every request is attributable to a scan and target scope.

• Deliverable: corrected scan pipeline and scanner-core regression tests.

Phase 4 — False-Positive Reduction

• Test each detector against positive, negative and ambiguous cases.

• For XSS, distinguish reflection from executable JavaScript and account for encoding/context.

• For SQL injection, distinguish harmless database/error strings from evidence of injection and use safe differential confirmation where appropriate.

• For path traversal, require evidence that the application accessed unintended filesystem content rather than merely reflecting a traversal string.

• For security headers, distinguish missing/weak headers from contextually meaningful security exposure.

• Add confidence levels: informational, suspected, probable, confirmed, with explicit evidence requirements.

• Deliverable: detector-specific false-positive report and verification layer.

Phase 5 — False-Negative Reduction

• Build a controlled regression suite using intentionally vulnerable training applications.

• Test multiple input locations: query parameters, POST bodies, forms, JSON, headers and path segments where applicable.

• Test encoded payloads, alternate parameter names, redirects, nested paths and dynamic pages.

• Measure which known vulnerabilities are missed and why.

• Prioritise coverage gaps before adding more vulnerability categories.

• Deliverable: coverage matrix and false-negative remediation backlog.

Phase 6 — Finding Intelligence & Classification

• Normalize finding schema across every scanner.

• Implement deterministic deduplication using target, endpoint, parameter, vulnerability class and evidence fingerprints.

• Verify CWE mappings individually.

• Verify CVSS vectors and ensure severity is derived from demonstrated impact rather than detector name alone.

• Store request/response evidence safely with redaction of credentials, tokens and sensitive data.

• Generate confidence and verification status independently from severity.

• Deliverable: canonical finding model and trustworthy severity/evidence pipeline.

Phase 7 — ML / RAG / AI Verification

• Determine which ML/AI components are actually executable and integrated into the scan path.

• Trace input data, model calls, prompts, retrieval sources, outputs and failure handling.

• Separate deterministic scanner evidence from AI-generated interpretation.

• Test hallucination, contradictory evidence, irrelevant retrieval and prompt-injection resistance.

• Do not report research-paper metrics as project metrics unless independently reproduced.

• Deliverable: verified AI architecture, evaluation set and AI limitations report.

Phase 8 — Reporting & Persistence

• Verify SQLite schema, transaction handling, concurrent access and scan isolation.

• Compare UI findings, database records, JSON/HTML/PDF outputs and any SARIF/JUnit outputs for consistency.

• Ensure reports preserve evidence and timestamps and clearly distinguish confirmed from suspected findings.

• Verify failed scans produce useful diagnostics rather than silently incomplete reports.

• Deliverable: consistent auditable report pipeline.

Phase 9 — Security Hardening

• Audit CENTRIX APIs for authentication/authorization requirements appropriate to deployment.

• Review SSRF-like behaviour, redirect abuse, DNS rebinding, private-address access, localhost access and cloud metadata exposure.

• Review command execution, file writes, archive handling, report generation and dependency risk.

• Ensure target-safety controls cannot be bypassed through alternate URL forms, redirects or DNS changes.

• Remove hard-coded secrets and ensure logs do not leak credentials or tokens.

• Deliverable: CENTRIX self-security assessment and hardened safety gate.

Phase 10 — Performance & Concurrency

• Measure request throughput, memory use, browser-process usage and database load on controlled targets.

• Test concurrent scans and ensure one scan cannot corrupt or contaminate another.

• Verify timeout and cancellation behaviour.

• Prevent uncontrolled crawling or request amplification.

• Deliverable: performance baseline, limits and concurrency fixes.

Phase 11 — End-to-End Validation

• Run the complete system against authorised vulnerable training targets.

• Use known expected vulnerabilities and record expected vs actual results.

• Calculate precision, recall, false-positive rate and false-negative rate per detector where the dataset is large enough.

• Verify evidence and reproduction steps for confirmed findings.

• Deliverable: final validation dataset and acceptance test report.

Phase 12 — Release Readiness

• Freeze the architecture and remove unsupported claims.

• Update README and technical documentation to reflect actual capabilities.

• Add automated CI tests for core scanner modules and regression targets.

• Define explicit limitations and future scope.

• Create a final release checklist and acceptance gate.

• Deliverable: CENTRIX release candidate with evidence-backed documentation.

4. Defect Classification

Priority	Meaning	Examples
P0	Critical	Safety boundary bypass, severe CENTRIX vulnerability, destructive unintended scanning, data loss
P1	Major	Major false results, broken scanner stage, incorrect severity, authentication/security failure
P2	Moderate	Coverage gap, unreliable edge case, report inconsistency, performance defect
P3	Minor	UI/documentation issue, code quality, non-critical usability problem

5. Required Finding Format

Finding ID → Component → File/Function → Severity → Confidence → Expected behaviour → Actual behaviour → Reproduction → Evidence → Root cause → Impact → Fix → Regression test → Verification status.

A scanner result should never be accepted merely because a payload was reflected or a keyword matched. The remediation goal is to make each finding explainable and reproducible.

6. Acceptance Gates

Gate	Pass condition
G1 — Build	Clean setup succeeds from documented instructions.
G2 — Runtime	Backend/frontend/scanner execute without known blocking exceptions.
G3 — Safety	Out-of-scope/private targets are consistently rejected or controlled.
G4 — Detection	Known vulnerable cases are detected with documented evidence.
G5 — Precision	False positives are measured and unacceptable detector behaviour is fixed.
G6 — Evidence	Confirmed findings contain reproducible evidence and provenance.
G7 — Classification	CWE/CVSS/severity are consistent with demonstrated conditions.
G8 — Reporting	UI/database/exported reports agree.
G9 — Regression	Automated tests prevent repaired bugs from returning.
G10 — Release	Documentation accurately describes only verified capabilities.

7. Current Known Documentation-Level Gaps

From the project material already available, the documented future/limitation areas include SQLite scaling, lack of authenticated scans, OpenAPI import, richer evidence capture, PostgreSQL/Redis-based scaling and additional integration testing. These are **baseline observations from documentation**, not yet source-code-confirmed defects.

The supplied project paper also contains quantitative evaluation claims (for example detection, throughput, hallucination and latency figures). These must be treated as reported-paper results until the underlying experiment, dataset, code version and methodology are independently reproduced.

8. Immediate Next Step

Attach the extracted CENTRIX ZIP to this conversation. The next audit pass will replace assumptions with direct source evidence and produce the first concrete defect register: file/function, exact problem, severity, reproduction procedure, expected result, actual result and proposed correction.

9. Final Target Architecture

Safe Target Validation → Discovery → Controlled Crawl → Deterministic Detection → Evidence Verification → Finding Normalisation → Optional AI/RAG Enrichment → Confidence/Severity → Persistent Storage → Auditable Reporting

The central engineering objective is not to maximise the number of alerts. It is to maximise the number of **correct, reproducible, explainable findings** while preventing unsafe or out-of-scope behaviour.